

PharmacieConnect

Dossier technique approfondi — Soutenance de PFE

Pharmacie de Garde & Assistance d'urgence

FastAPI · Backend

React + Vite · Admin

Expo React Native · Mobile

RAG local · Chatbot IA

Auteur : Mohamed Teyeb

Approfondissement : Authentification JWT · Import CSV · Chatbot RAG · RBAC régional

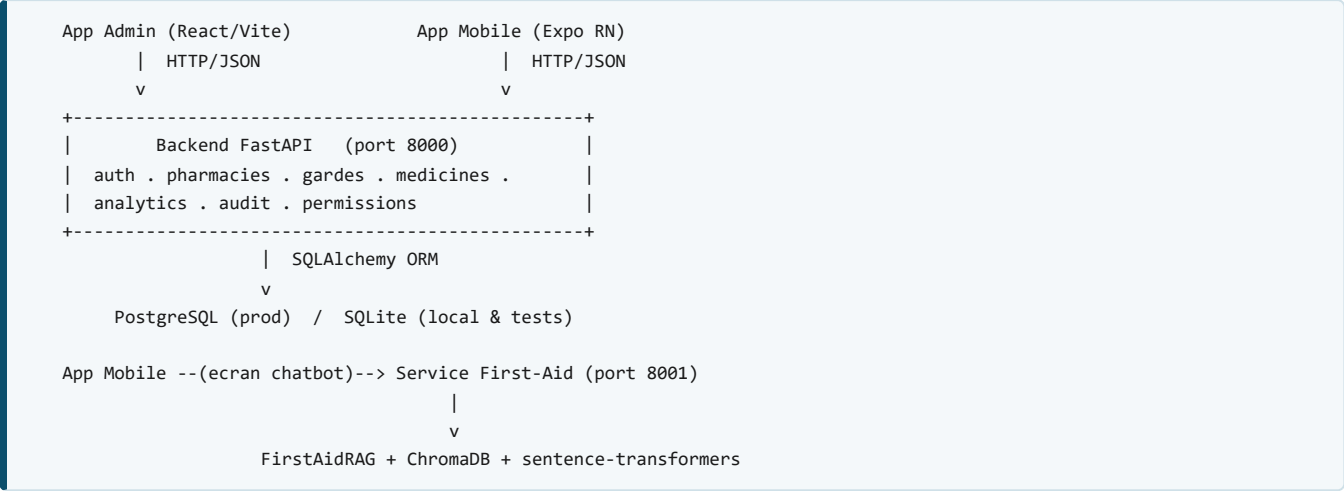
Document généré pour la préparation de la soutenance technique

Sommaire

1. Rappel de l'architecture (le contexte des approfondissements)
2. **Approfondissement 1 — Authentification & Sécurité JWT**
3. **Approfondissement 2 — Autorisation par rôle & portée régionale (RBAC)**
4. **Approfondissement 3 — Import CSV (Pharmacies & Gardes)**
5. **Approfondissement 4 — Le Chatbot Premiers Secours (RAG local)**
6. Approfondissement 5 — Recherche géospatiale & Cache
7. Approfondissement 6 — Frontend Admin & rafraîchissement de token
8. Synthèse : questions probables du jury

1. Rappel de l'architecture

Le système est composé de **4 applications indépendantes** qui communiquent en HTTP/JSON. Le backend est la **source unique de vérité**.



Composant	Rôle	Stack	Port
backend_pharmacie	API REST centrale, BD, sécurité	FastAPI + SQLAlchemy	8000
admin_pharmacie	Tableau de bord administrateur	React + Vite	5173
mobile	App citoyenne	Expo React Native	—
firstaid-ai	Chatbot premiers secours (IA)	FastAPI + ChromaDB	8001

1.1 Diagramme de cas d'utilisation (Use Case)

Quatre acteurs interagissent avec le système. Le **citoyen** utilise l'app mobile publique ; les **comptes du back-office** (assistant régional, administrateur, super admin) gèrent les données opérationnelles avec des permissions croissantes.

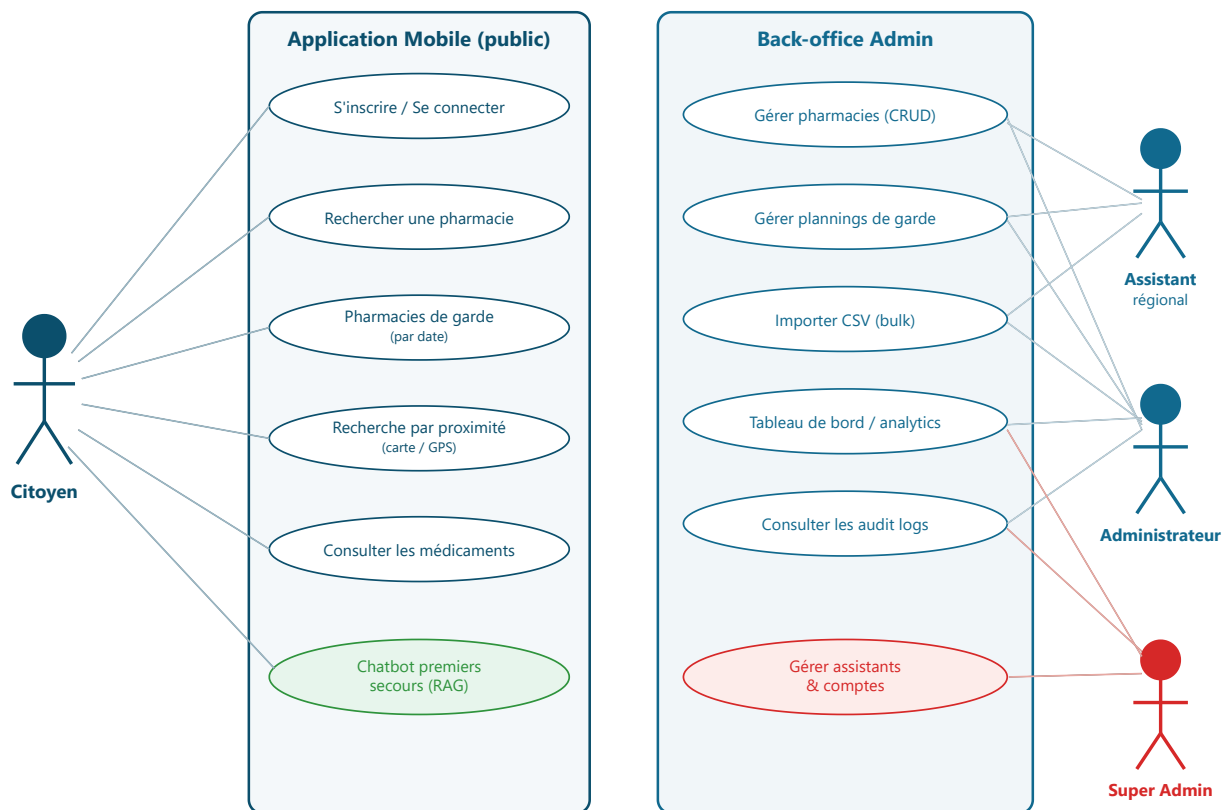


Fig. 1 — Diagramme de cas d'utilisation. Le super admin hérite des droits de l'admin (*) ; l'assistant est limité aux opérations régionales.

Architecture en couches du backend

Requête HTTP

- > Router (routers/) traduit HTTP <-> appel de service
- > Dependency (dependencies.py) valide le token JWT + le rôle
- > Service (services/) la VRAIE logique métier
- > SQLAlchemy Session accès base de données
- > Base de données

🎯 **À DIRE AU JURY** — « Les routers sont **minces** : ils ne font que traduire une requête HTTP en appel de service. Toute la logique métier — validation, parsing CSV, règles, audit — est isolée dans les *services*. Ça rend le code testable et maintenable. »

1.2 Architecture du code (vue par couches & fichiers réels)

Chaque requête traverse les couches de haut en bas. Les noms ci-dessous sont les **fichiers réels** du dépôt.

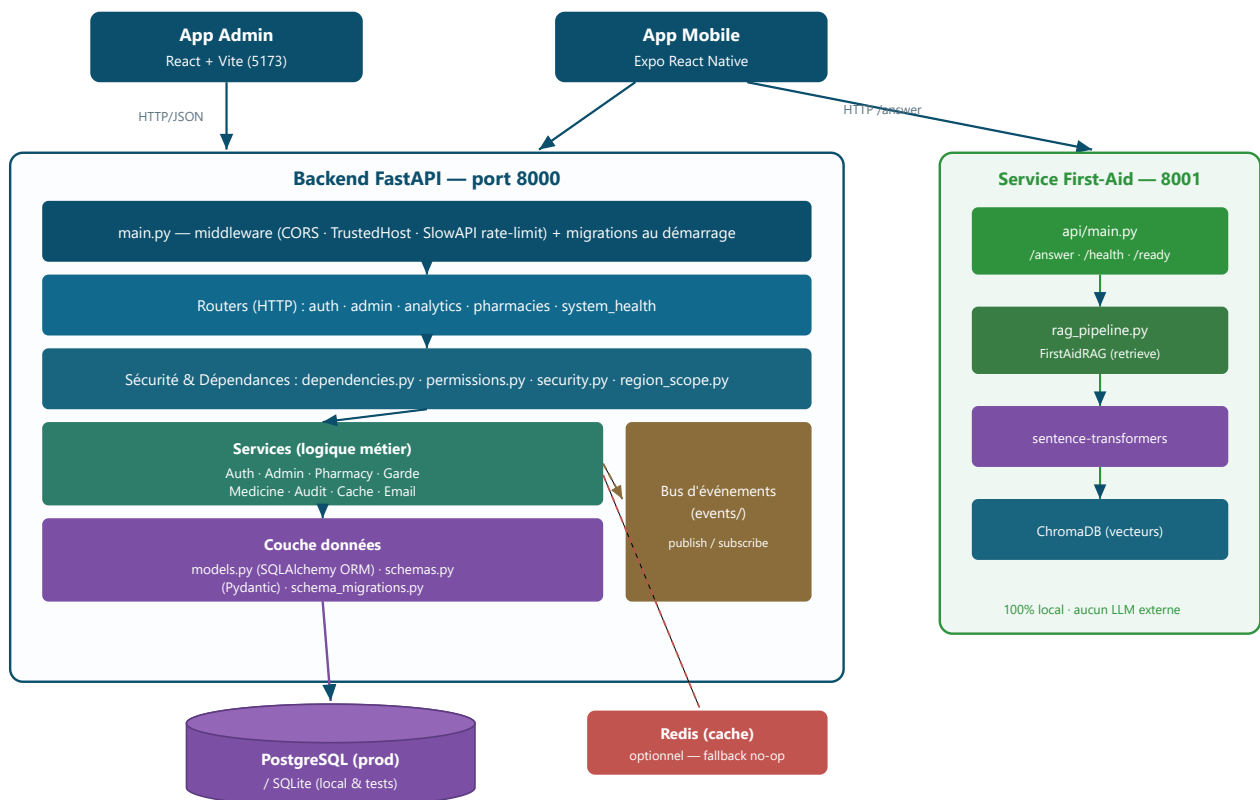


Fig. 2 — Architecture du code. Flux d'une requête de haut en bas. Le service First-Aid est isolé du backend principal.

1.3 Diagramme de classes (modèle de données)

9 tables SQLAlchemy. `Administrateur` est le **hub** : il « possède » (`created_by`) les pharmacies, gardes et médicaments. `RefreshToken`, `LoginAttempt` et `AuditLog` sont **polymorphes** (champ `entity_type` qui pointe vers admin OU utilisateur, sans clé étrangère stricte).

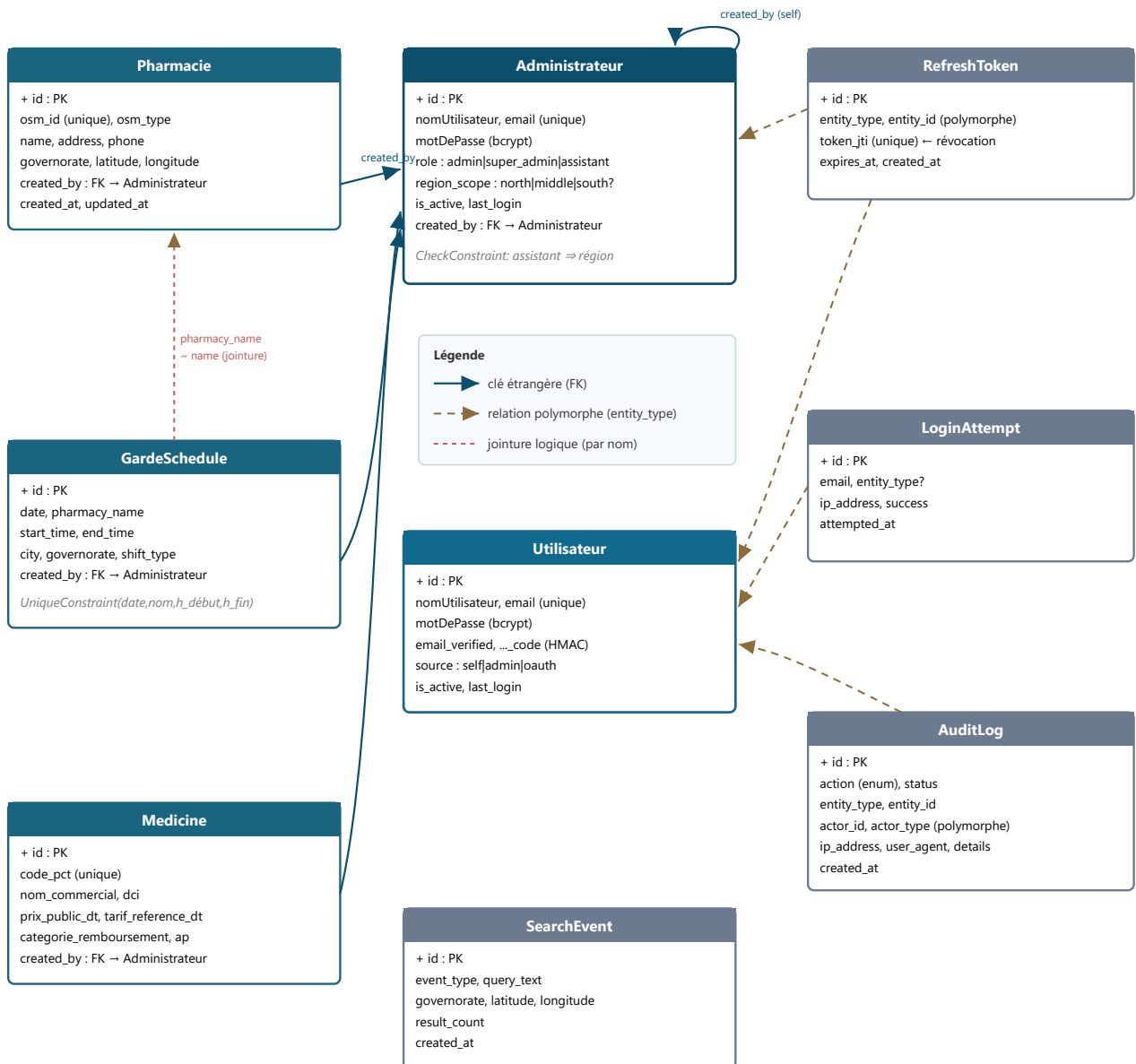


Fig. 3 — Diagramme de classes. Admins et utilisateurs sont volontairement dans deux tables distinctes ; les tables d'audit/sécurité les référencent par `entity_type` + `entity_id`.

2. Approfondissement 1 — Authentification & Sécurité JWT

Fichiers : security.py · services/auth_service.py · dependencies.py · routers/auth.py

2.1 Le hachage des mots de passe (bcrypt)

Les mots de passe ne sont **jamais stockés en clair**. On utilise **bcrypt** avec un coût de 12 tours (`BCRYPT_ROUNDS = 12`). Bcrypt génère un *sel aléatoire* intégré au hash, donc deux utilisateurs avec le même mot de passe ont des hash différents.

```
def hash_password(password: str) -> str:
    return bcrypt.hashpw(password.encode("utf-8"),
                          bcrypt.gensalt(rounds=BCRYPT_ROUNDS)).decode("utf-8")

def verify_password(plain, hashed) -> bool:
    return bcrypt.checkpw(plain.encode(), hashed.encode()) # comparaison constante
```

2.2 Le système à deux tokens (Access + Refresh)

Token	Durée de vie	Contenu (claims)	Stockage backend
Access token	15 min (<code>ACCESS_TOKEN_EXPIRE_MINUTES</code>)	<code>sub</code> , <code>role</code> , <code>entity_type</code> , <code>type=access</code> , <code>iss</code> , <code>aud</code> , <code>iat</code> , <code>exp</code>	Aucun (stateless)
Refresh token	7 jours (<code>REFRESH_TOKEN_EXPIRE_DAYS</code>)	<code>sub</code> , <code>entity_type</code> , <code>type=refresh</code> , <code>jti</code> (id unique), ...	Son <code>jti</code> stocké en table <code>refresh_tokens</code>

Algorithme : **HS256**, signé avec `SECRET_KEY`. Le backend **refuse de démarrer** si la clé fait moins de 32 caractères ou contient une valeur par défaut — sécurité dès la configuration :

```
if not SECRET_KEY or len(SECRET_KEY) < 32 or "change-this" in SECRET_KEY.lower():
    raise RuntimeError("SECRET_KEY must be set to a strong secret of at least 32 characters")
```

Pourquoi 2 tokens ? L'access token est court → s'il est volé, la fenêtre d'attaque est minime. Le refresh token, lui, est long mais **révocable** côté serveur (voir 2.4). On combine le confort (pas de reconnexion toutes les 15 min) et la sécurité (révocation possible).

2.3 Le flux de connexion (`AuthService.login`)

1. POST `/api/auth/login` { email, password }
2. Recherche d'abord dans la table `administrateurs`
3. Sinon cherche dans la table `utilisateurs`
4. `verify_password(password, hash)` -- bcrypt
5. (utilisateur) : vérifie que l'email est confirmé (`email_verified`)
6. `create_access_token()` + `create_refresh_token()`
7. Enregistre le `jti` du refresh dans `refresh_tokens`
8. Enregistre un `LoginAttempt` + un `AuditLog` (succès/échec)
9. Retourne { `access_token`, `refresh_token`, `token_type`, `expires_in` }

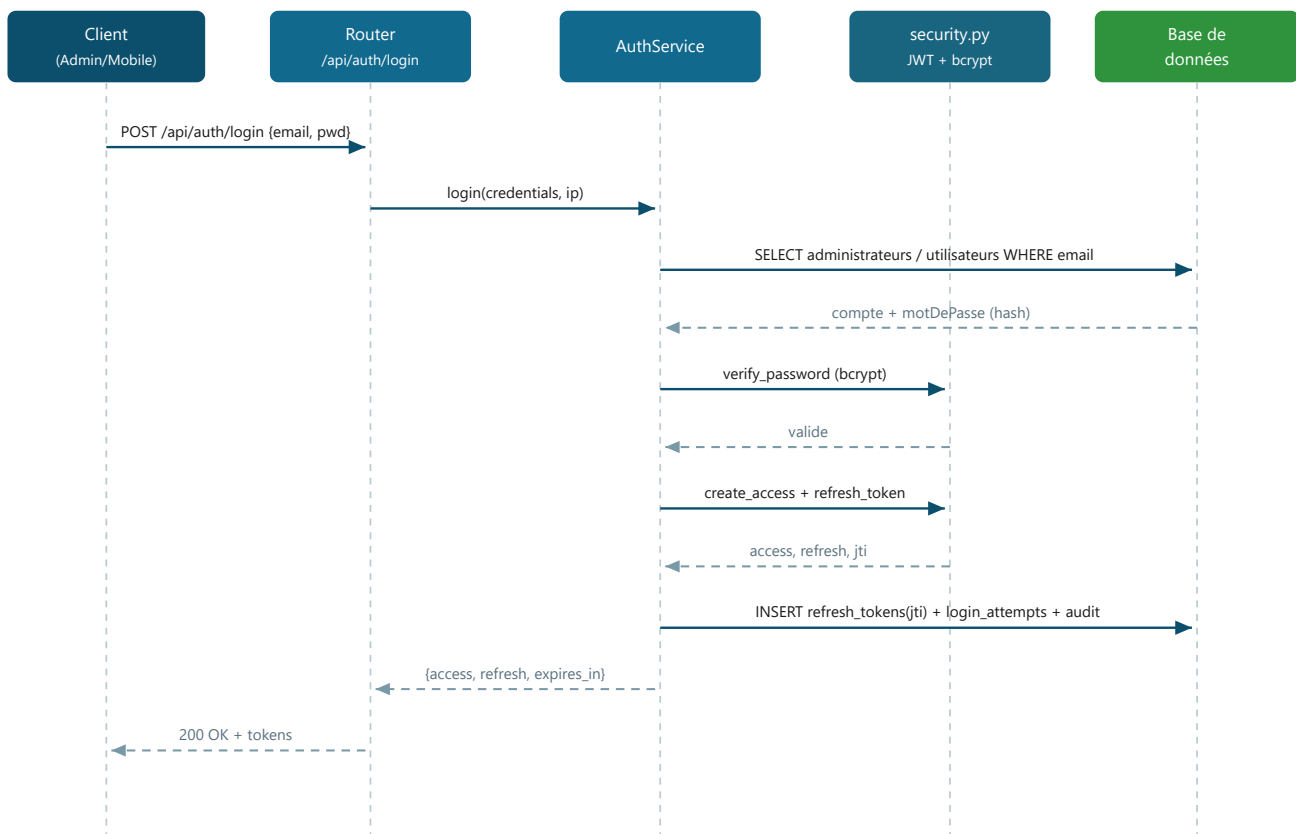


Fig. 4 — Séquence de connexion. Flèches pleines = appels ; flèches pointillées = retours. Le **jti** du refresh est persisté (révocabilité).

Détail important : le champ **entity_type** (**administrateur** ou **utilisateur**) est inscrit **dans le token**. Il détermine dans quelle table chercher le compte → c'est la clé de la séparation admin/utilisateur.

2.4 Refresh & Logout — la révocation

Quand l'access token expire, le client appelle **/api/auth/refresh** avec le refresh token. Le backend vérifie que le **jti** **existe encore** en base :

```

payload = verify_token(refresh_token, token_type="refresh")
revoked = db.query(RefreshToken).filter(RefreshToken.token_jti == jti).first()
if not revoked:
    # le jti n'est plus en base
    return None, "Refresh token was revoked"
new_access_token = create_access_token(user.id, role, entity_type)
  
```

Au **logout**, on **supprime la ligne** du **jti** en base. Le refresh token devient alors inutilisable, même s'il n'a pas expiré :

```
db.query(RefreshToken).filter(RefreshToken.token_jti == jti).delete()
```

🔗 **À DIRE AU JURY** — « Un JWT classique est *stateless* donc impossible à révoquer. Ici, en stockant uniquement le **jti** du refresh token en base, je rends la déconnexion réellement effective : c'est le meilleur des deux mondes entre *stateless* et révocabilité. »

2.5 La validation à chaque requête (dependencies.py)

FastAPI injecte des **dépendances** qui s'exécutent avant le code de la route. **verify_token()** vérifie la signature, l'expiration, l'émetteur (**iss**), l'audience (**aud**) et le type de token :


```

async def get_current_admin(authorization=Header(None), access_token=Cookie(None), db=...):
    token = _extract_bearer_or_cookie_token(authorization, access_token) # header OU cookie
    payload = verify_token(token, token_type="access")
    admin_id = _validate_subject(payload, "administrateur") # 403 si mauvais entity_type
    admin = db.query(Administrateur).filter_by(id=admin_id).first()
    if not admin or not admin.is_active: raise HTTPException(401)
    return admin

```

Le token est accepté depuis l'en-tête `Authorization: Bearer ...` (mobile/admin) **ou** un cookie (web) → souplesse multi-client.

2.6 Récapitulatif des protections de sécurité

- Mots de passe hachés bcrypt (sel + 12 tours)
- Access token court (15 min), refresh token révocable (jti en base)
- Claims `iss` / `aud` / `type` vérifiés → un refresh token ne peut pas servir d'access token
- `SECRET_KEY` obligatoire et forte au démarrage
- Codes de vérification email hachés (HMAC-SHA256), expiration 15 min, max 5 tentatives
- Comparaison constante (`secrets.compare_digest`) → anti timing-attack
- Table `login_attempts` + `audit_logs` → traçabilité et détection d'intrusion
- Rate limiting (SlowAPI), CORS strict, TrustedHost (anti Host-header injection)

3. Approfondissement 2 — Autorisation par rôle & portée régionale

Fichiers : `permissions.py` · `dependencies.py` · `region_scope.py` · `models.py`

3.1 Les rôles (RBAC)

Rôle	Permissions	Dépendance FastAPI
<code>super_admin</code>	<code>*</code> (tout)	<code>super_admin_required</code>
<code>admin</code>	<code>*</code> (tout)	<code>admin_required</code>
<code>assistant</code>	lecture + écriture régionale seulement	<code>staff_required</code>
<code>user</code>	app mobile (consultation publique)	<code>get_current_user</code>

```
ADMIN_ROLES = {"admin", "super_admin"}
STAFF_ROLES = ADMIN_ROLES | {"assistant"}
ROLE_PERMISSIONS = {
    "assistant": {"pharmacies:read", "pharmacies:write:regional",
                  "gardes:read", "gardes:write:regional", "regions:read", ...},
    "admin": {"*"}, "super_admin": {"*"},
}
```

🔗 **À DIRE AU JURY** — « La protection n'est PAS dans le frontend — le frontend ne fait que cacher des boutons. La vraie protection est dans le backend : chaque route admin dépend de `admin_required`, `staff_required` ou `super_admin_required` qui vérifient le JWT et le rôle. »

3.2 La portée régionale des assistants

Un assistant a un champ `region_scope` ∈ { `north`, `middle`, `south` }. Les 24 gouvernorats tunisiens sont regroupés en 3 zones (`region_scope.py`). Toute requête est **filtrée automatiquement** au niveau SQL :

```
def apply_region_scope(query, model, region):
    values = region_filter_values(region) # ex: {"tunis","ariana","ben arous",...}
    if not values: return query           # admin -> aucun filtre
    return query.filter(func.lower(model.governorate).in_(values))
```

Et lors d'une écriture (création/import), `region_access_error()` rejette toute donnée hors zone :

```
def region_access_error(governorate, region):
    if not governorate: return "Governorate is required for assistant accounts ..."
    if not governorate_in_region(governorate, region):
        return f"Governorate '{governorate}' is outside the assistant region ..."
    return None
```

3.3 Intégrité garantie au niveau base de données

Une règle métier critique est protégée par une **contrainte SQL** (pas seulement par le code Python) : un assistant DOIT avoir une région, un non-assistant NE doit PAS en avoir.

```
CheckConstraint(
    "((role = 'assistant' AND region_scope IN ('north','middle','south')) "
    " OR (role <> 'assistant' AND region_scope IS NULL))",
    name="ck_administrateurs_assistant_region_scope")
```

Normalisation robuste : `normalize_text()` retire les accents (`Béja` → `beja` , `Gabès` → `gabes`) pour que le filtrage marche quel que soit l'encodage du CSV.

4. Approfondissement 3 — Import CSV (Pharmacies & Gardes)

Fichiers : `services/pharmacy_service.py` · `services/garde_service.py` (pandas)

4.1 Le pipeline de validation (10 étapes)

L'import utilise **pandas** pour lire le CSV puis applique une validation défensive, étape par étape :

1. Extension .csv ? → sinon rejet
2. Taille <= 5 Mo (MAX_FILE_SIZE) ? → sinon rejet
3. `pd.read_csv()` → sinon "Failed to parse"
4. Fichier non vide ?
5. Nb lignes <= 5000 (MAX_ROWS) ?
6. Colonnes requises présentes ? (résolution par ALIAS)
7. Pour chaque ligne : parse + valide + vérifie doublons + région
8. Au moins 1 ligne valide ?
9. Insertion en base (commit) -- rollback si erreur
10. AuditLog + événement publié

4.2 Tolérance aux formats réels (alias de colonnes)

Les fichiers réels n'ont jamais des noms de colonnes parfaits. Le service mappe des **alias** vers les champs canoniques :

```
COLUMN_ALIASES = {
    "latitude": {"latitude", "lat"},
    "longitude": {"longitude", "lon"},
    "governorate": {"governorate", "region", "province", "state"},
    "phone": {"phone", "tel", "telephone"}, ...
}
REQUIRED_COLUMNS = {"name", "latitude", "longitude"}
```

Chaque ligne en erreur n'arrête PAS l'import : elle est collectée avec son **numéro de ligne** et renvoyée au frontend. L'utilisateur voit exactement quelles lignes corriger.

```
return {"total_rows": N, "successful": ok, "failed": len(errors), "errors": errors}
```

4.3 Gardes : double format + dates françaises

`GardeService` détecte automatiquement **2 formats** :

- **Format standard** : colonnes `date`, `pharmacy_name`, `start_time`, `end_time`, ...
- **Format "planner"** (tableau de garde réel) : `category`, `month`, `date`, `pharmacist_1`, `pharmacist_2` — où la date est en français (« 12 janvier »)

```
FRENCH_MONTHS = {"janvier":1, "fevrier":2, "février":2, "mars":3, ... "décembre":12}

def _parse_planner_date(value):
    match = re.search(r"(\d{1,2})(?:\s*-\s*(\d{1,2}))?\s+([A-Za-zÀ-ÿ]+)", normalized)
    month = FRENCH_MONTHS.get(match.group(3).lower())
    return date(datetime.now().year, month, int(match.group(1)))
```

4.4 Anti-doublons : la stratégie "upsert"

Avant d'insérer, on cherche une garde identique (même date + pharmacie + horaires). Si elle existe → **mise à jour** ; sinon → insertion. Doublons aussi bloqués par un `UniqueConstraint` en base.

```
existing = db.query(GardeSchedule).filter(date==..., pharmacy_name==...,
                                          start_time==..., end_time==...).first()
if existing: existing.city = ...           # update
else:        db.add(GardeSchedule(...))  # insert
```

4.5 Sécurité de l'import

- Réservé aux comptes `staff_required` (admin/assistant)
- Un assistant ne peut importer que pour **sa région** (`region_access_error` par ligne)
- Transaction atomique : `commit()` global, `rollback()` en cas d'échec → pas d'état partiel
- Chaque import génère un `AuditLog` (qui, quand, combien de lignes)

Limite assumée : l'import est **synchrone**. Pour de très gros volumes, on le déplacerait vers une tâche d'arrière-plan (Celery / background task FastAPI). C'est une évolution prévue.

5. Approfondissement 4 — Chatbot Premiers Secours (RAG local)

Fichiers : firstaid-ai/rag_pipeline.py · firstaid-ai/api/main.py (port 8001)

5.1 Qu'est-ce que le RAG ici ?

RAG = **Retrieval-Augmented Generation**. Mais ici c'est un **RAG « pur » : recherche sémantique sans génération par LLM**. On ne renvoie que des passages **validés** du dataset → **pas d'hallucination**, et aucune dépendance à une API externe payante.

Contrainte projet respectée : aucune dépendance à un LLM externe (pas de Claude / Ollama / Groq / HuggingFace Inference au runtime). 100% local : `sentence-transformers` pour l'embedding + `ChromaDB` pour le stockage vectoriel.

5.2 Les briques techniques

Brïque	Rôle
sentence-transformers paraphrase-multilingual-MiniLM-L12-v2	Transforme un texte en vecteur (embedding) de sens. Modèle multilingue : français, anglais, arabe .
ChromaDB	Base de données vectorielle persistante. Stocke tous les passages de premiers secours + leurs métadonnées.
Distance cosinus	Mesure de proximité de sens entre la question et les passages.

5.3 Le flux d'une question (POST /answer)

1. Mobile -> POST `http://host:8001/answer { query, top_k }`
2. `_expand_query()` : injecte 1 synonyme-pont si besoin (multilingue)
3. `embedder.encode(query)` -> vecteur normalisé
4. `ChromaDB.query()` : récupère `top_k*4` candidats (pool)
5. `retrieve()` : re-classement hybride (`final_score`)
6. seuils de confiance + détection d'urgence
7. Réponse adaptée : `{ answer, confidence, answer_mode, topic_prediction, retrieved_evidence }`

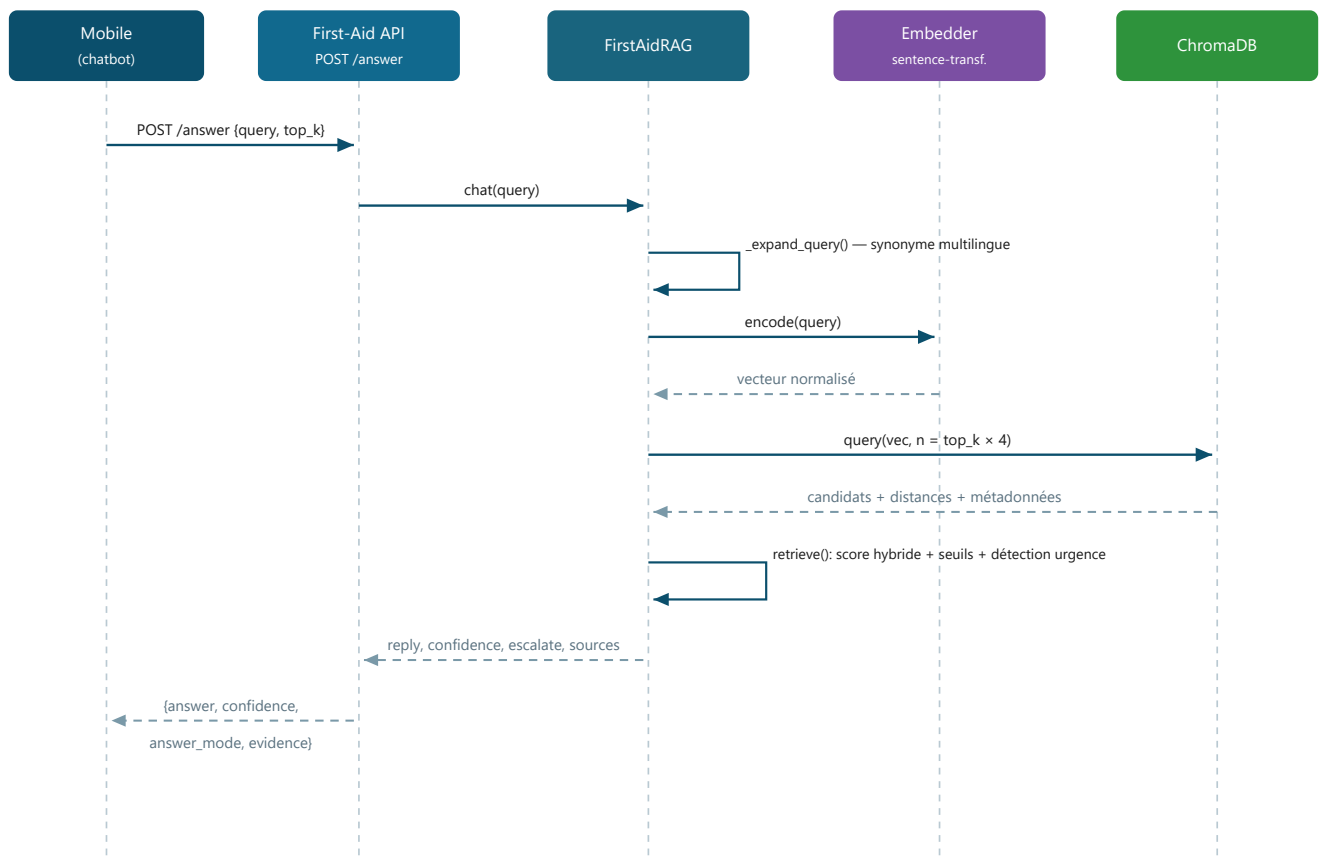


Fig. 5 — Séquence du chatbot RAG. Aucun appel à un LLM externe : encodage local + recherche vectorielle ChromaDB + re-classement hybride.

5.4 Le re-classement hybride (le cœur intelligent)

On ne se fie pas qu'à la distance cosinus. Chaque candidat reçoit un **score composite** combinant 4 signaux :

```

final_score = 0.60 * cos_sim      # proximité sémantique
              + 0.20 * quality_boost # qualité de la source (1/2/3)
              + 0.10 * has_steps_boost # le passage contient des étapes d'action ?
              + 0.10 * age_match_boost # adapté à l'âge demandé (bébé/enfant/adulte) ?
  
```

Puis on applique 3 règles métier :

- **Plancher de qualité** : un passage de source faible (quality=1) ne peut pas être en position 1.
- **Diversité** : si le top-3 vient de la même source, on remplace le 3^e par un autre.
- **Filtre par catégorie** : si le top-1 est très sûr (`cos_sim ≥ 0.80`), on privilégie sa catégorie.

5.5 Sécurité médicale : seuils & détection d'urgence

Distance cosinus	Interprétation
< 0.45	Haute confiance
< 0.65	Confiance moyenne
> 0.72	Hors-domaine → refus de répondre (REFUSAL_REPLY)

Détection d'urgence trilingue (`URGENCY_TRIGGERS`) : « ne respire pas », « inconscient », « heavy bleeding », nécessité d'**escalade** immédiate vers les secours (SAMU 190 en Tunisie). Une couche de règles symptomatiques (`SYMPTOM_RULES`) couvre les cas critiques (arrêt cardiaque, AVC, anaphylaxie, étouffement...) avec des étapes d'action vérifiées.

Support multilingue concret : la question peut être posée en français, anglais ou arabe (ex. « طفل يحترق، ماذا أفعل؟ »). Le module détecte le script (`_detect_script`) et adapte synonymes et expansion.

5.6 Chiffres d'évaluation à citer

Métrique	Valeur
Exemples de validation / test	310 / 310
Précision top-1 du topic	≈ 95,48 %
Taux de réussite top-5	≈ 97,74 %
Taux de violation de sécurité	0,0

🔗 **À DIRE AU JURY** — « L'assistant fonctionne 100% en local au runtime, sans API externe payante. C'est de la recherche sémantique pure donc **pas d'hallucination** : il ne renvoie que des passages validés. Et c'est une *aide* aux premiers secours qui ne remplace jamais les services d'urgence — chaque réponse porte un disclaimer et escalade en cas d'urgence détectée. »

5.7 Sources des données (le corpus du chatbot)

Le corpus brut compte ≈ **7 000 paires question/réponse** de premiers secours (`firstaid-ai/data/firstaid_raw.jsonl`). Il est construit par `fetch_dataset.py` à partir de **jeux de données médicaux publics (Hugging Face)** filtrés par mots-clés premiers secours, complétés par des **entrées rédigées à la main** (« seeds » curées) en anglais, français et arabe.

Source	Type	Entrées	Description
<code>medalpaca/medical_meadow_medical_flashcards</code>	HF public	3 414	Fiches médicales Q/R
<code>qiaojin/PubMedQA</code>	HF public	1 438	Q/R biomédicales issues de PubMed
<code>openlifescienceai/medmcqa</code>	HF public	1 303	QCM médicaux (avec explications)
<code>GBaker/MedQA-USMLE-4-options</code>	HF public	384	Questions d'examen médical USMLE
<code>lavita/ChatDoctor-HealthCareMagic-100k</code>	HF public	211	Dialogues patient-médecin
<code>BI55/MedText</code>	HF public	41	Textes médicaux
<code>curated/seed</code>	Rédigé main	20	Gestes de premiers secours validés (EN/FR)
<code>curated/arabic/*</code>	Rédigé main	≈ 198	20 catégories d'urgence en arabe (brûlure, étouffement, AVC...)

Pipeline de construction

- `fetch_dataset.py` -> télécharge les datasets HF (`load_dataset`)
- Filtre mots-clés -> garde uniquement les lignes "premiers secours" (burn, bleeding, CPR, choking, stroke, seizure...)
- Filtre urgence -> 2e filtre strict pour les corpus d'urgence
- Seeds curées -> ajoute les entrées rédigées à la main (EN/FR/AR)
- Déduplication -> `difflib.SequenceMatcher`, seuil 0.90 -> retire les quasi-doublons
- `firstaid_raw.jsonl` -> ~7000 entrées uniques
- `split` -> train / validation (310) / test (310)
- `embed_and_store.py` -> vectorise (`sentence-transformers`) -> ChromaDB

Chaque entrée est enrichie de **métadonnées** : `category` (burn, cardiac, neuro, airway...), `severity_hint` (high / standard) et `source` — ces métadonnées alimentent le score hybride (§5.4) et la détection d'urgence.

Licences & éthique : toutes les sources externes sont des jeux de données **publics et ouverts** (Hugging Face). Le téléchargement n'a lieu qu'**une seule fois**, hors ligne, au moment de la construction du corpus ; **au runtime, rien n'est téléchargé** — le chatbot interroge uniquement le ChromaDB local. Les gestes critiques (RCP, étouffement, hémorragie...) proviennent des **seeds rédigées et vérifiées à la main**, alignées sur les protocoles de premiers secours.

 **À DIRE AU JURY** — « Les données viennent de jeux médicaux publics (Hugging Face : MedAlpaca, PubMedQA, MedMCQA, MedQA-USMLE...) filtrés sur les premiers secours, plus ~220 entrées que j'ai rédigées et vérifiées à la main en français, anglais et arabe pour couvrir les gestes critiques. Tout est déduplicé, vectorisé une seule fois, puis stocké localement dans ChromaDB — aucune donnée n'est récupérée pendant l'utilisation. »

6. Approfondissement 5 — Recherche géospatiale & Cache

6.1 Recherche de pharmacies « à proximité » (formule de Haversine)

L'endpoint `/api/pharmacies/nearby` calcule la distance entre l'utilisateur et chaque pharmacie avec la **formule de Haversine** (distance sur une sphère), directement en SQL :

```
R = 6371 # rayon de la Terre en km
distance = R * acos( cos(lat1 - lat2) * cos(lon1 - lon2)
                    + sin(lat1 - lat2) * sin(lon1 - lon2) )
query.filter(distance <= radius_km).order_by(distance).limit(limit)
```

Limite assumée : pour la production à grande échelle, **PostGIS** (index géospatial GiST) serait plus performant qu'un calcul trigonométrique par ligne. Le choix actuel reste simple et suffisant pour le volume du projet.

6.2 Cache avec dégradation gracieuse (`CacheService`)

Les endpoints publics lourds (gardes, médicaments) sont mis en cache JSON via Redis. Mais Redis est **optionnel** : si indisponible, le cache devient un *no-op* et l'app continue de fonctionner.

```
cache_key = f"gardes:list:{date.isoformat()}:{skip}:{limit}"
cached = cache_service.get_json(cache_key)
if cached is not None: return cached # HIT
payload = garde_service.get_public_gardes(...) # MISS -> calcul
cache_service.set_json(cache_key, payload, ttl_seconds=900) # TTL 15 min
```

6.3 Autres mécanismes de fiabilité

- **Pagination** partout (`skip` / `limit` , plafonné à 500)
- **Migrations idempotentes** au démarrage (`run_schema_migrations`)
- **Bus d'événements** interne : chaque action importante publie un événement (découplage, extensibilité)
- **Connection pooling** PostgreSQL

7. Approfondissement 6 — Frontend Admin & rafraîchissement de token

Fichiers : `admin_pharmacie/src/lib/api.js` · `context/AuthContext.jsx` · `App.jsx`

7.1 L'intercepteur Axios (transparence du refresh)

L'instance Axios ajoute automatiquement le `Bearer token` à chaque requête. Si une réponse renvoie **401** (token expiré), un intercepteur déclenche le flux de refresh, puis **réessaie** la requête d'origine — sans que l'utilisateur ne voie rien.

```
Requête -> 401 Unauthorized
  -> intercepteur : POST /api/auth/refresh (refresh token)
    -> succès : nouveau access token -> on REJOUÉ la requête
    -> échec : on efface les tokens -> redirection /login
```

7.2 Protection des routes côté React

- `/login` est publique
- Les autres routes passent par `<ProtectedRoute>` (vérifie l'auth via `AuthContext`)
- Certaines pages exigent un rôle admin (dashboard, upload médicaments, audit logs)
- Les assistants accèdent aux pages staff, mais le **filtrage régional reste appliqué côté backend**

🗣️ **À DIRE AU JURY** — « Le frontend gère l'expérience (cacher les boutons, rediriger). Mais je ne fais jamais confiance au frontend pour la sécurité : toute autorisation est revérifiée côté backend. »

7.3 Côté mobile (Expo React Native)

- Navigation conditionnelle : non-connecté → pile login/register/vérification ; connecté → onglets (Home, Map, Calendar, Medicines, Settings) + Chatbot
- **Context providers** : auth, theme, language (i18next), favorites, filters, notifications, ratings, history
- **Data loaders** : `pharmacyDataLoader.js`, `medicineDataLoader.js`, `chatbotApi.js`
- Config : backend port 8000, chatbot port 8001, surchargeables via `EXPO_PUBLIC_API_URL` / `EXPO_PUBLIC_CHATBOT_API_URL`

8. Synthèse — Questions probables du jury

Pourquoi FastAPI ?

Documentation auto (Swagger/OpenAPI), validation Pydantic, injection de dépendances, performances élevées (async). Idéal pour une API REST modulaire en Python.

Pourquoi séparer admins et utilisateurs en 2 tables ?

Responsabilités et règles de sécurité différentes. Le champ `entity_type` dans le JWT route vers la bonne table → contrôles de rôle plus clairs et plus sûrs.

Comment révoquer un JWT alors qu'il est stateless ?

On ne révoque que le **refresh token** : son `jti` est stocké en base. Au logout, on supprime la ligne. L'access token, lui, est volontairement très court (15 min).

Que se passe-t-il si l'access token expire pendant l'utilisation ?

L'intercepteur Axios appelle `/refresh`, obtient un nouveau token et réessaie la requête. Si le refresh échoue → déconnexion + redirection login.

Comment un assistant est-il limité à sa région ?

Champ `region_scope`. En lecture, `apply_region_scope` ajoute un filtre SQL ; en écriture, `region_access_error` rejette les gouvernorats hors zone. Une contrainte SQL garantit l'intégrité.

Le chatbot utilise-t-il ChatGPT / un LLM externe ?

Non. C'est un RAG **local pur** : sentence-transformers + ChromaDB. Recherche sémantique sur des passages validés, donc pas d'hallucination et pas d'API payante.

Pourquoi un service chatbot séparé ?

Dépendances ML lourdes + comportement runtime différent. L'isoler garde le backend principal léger et rend le service IA déployable indépendamment.

Comment fonctionne la recherche « à proximité » ?

Formule de Haversine en SQL à partir des coordonnées lat/long, tri par distance croissante, filtré par rayon.

Comment l'import CSV gère les erreurs ?

Validation par étapes (extension, taille, lignes, colonnes par alias). Chaque ligne invalide est collectée avec son numéro et renvoyée ; les lignes valides sont insérées en transaction atomique ; un AuditLog est créé.

Quelles sont les limites du projet ?

- Imports CSV synchrones → à passer en tâches d'arrière-plan pour de gros volumes
- Géospatial par calcul trigonométrique → PostGIS en production
- Chatbot = aide aux premiers secours, jamais un substitut médical
- Redis optionnel (cache no-op si absent)

Script de défense (30 secondes)

 **À DIRE AU JURY** — « PharmacieConnect est un système d'accès aux pharmacies et d'assistance d'urgence : un backend FastAPI (source de vérité), un dashboard admin React, une app mobile Expo et un chatbot premiers secours RAG local. L'architecture est en couches — routers, dépendances d'auth/rôle, services métier, SQLAlchemy → PostgreSQL. La sécurité repose sur bcrypt, des JWT à access court et refresh révocable, l'autorisation par rôle avec portée régionale, le rate limiting et l'audit. Le chatbot fait de la recherche vectorielle locale avec ChromaDB et sentence-transformers, donc sans API externe. »